SABI

TECHNICAL AUDIT & DEVELOPER HANDOVER

AI-Powered Adaptive JAMB UTME Preparation Platform

Repository: github.com/sabiprep/sabi

Branch: main

Document status: Audit completed; implementation and production verification pending

November 2026

1. Executive Summary

Sabi already has a substantial technical foundation. The repository contains a React frontend, Express backend, Supabase database architecture, learning services, adaptive question-selection logic, AI provider integrations, administrative functionality and supporting platform features.

The recommendation is not to rebuild Sabi. The priority is to preserve existing working code, consolidate overlapping implementations, complete unfinished functionality and verify that the core learning experience works from beginning to end.

Sabi's central promise: Help students identify their weaknesses, master difficult topics, practise intelligently and improve their UTME performance.

2. Product Scope: What Sabi Must Deliver

2.1 Core learner experience

1. Student registration, authentication and onboarding.
2. Subject selection and learning goals.
3. Diagnostic assessment.
4. Personalised learning dashboard.
5. SmartPractice – adaptive question selection.
6. Blitz – continuous personalised question practice.
7. CBT simulator – realistic examination experience.
8. Mastery tracking and weak-topic detection.
9. Spaced repetition and revision scheduling.
10. Progress analytics.
11. UTME score prediction on a 0–400 scale.
12. Sabi AI Coach – personalised explanations and tutoring.

2.2 Essential administrative capabilities

- Question-bank management and moderation.
- JAMB syllabus and topic management.
- User and access management.
- AI provider and model configuration.
- AI usage, cost and budget monitoring.
- Basic platform health monitoring.

2.3 Features to defer

Do not allow supporting features to distract from the core learning experience. Defer or limit extensive parent portals, school and tutorial-centre portals, advanced administrative command centres, excessive gamification, unnecessary social features and additional dashboards that do not improve learning outcomes.

Billing, notifications, missions, streaks and rewards can remain supporting features where already functional.

3. Existing Technical Architecture

Layer	Existing technology
Frontend	React 19, Vite 6, TypeScript
Routing	React Router 7
Styling	Tailwind CSS 4

Layer	Existing technology
Backend	Express 4
Database	Supabase / PostgreSQL
Authentication	Supabase Auth
AI integration	Multi-provider gateway
Mathematical rendering	KaTeX
Mobile integration	Capacitor
Deployment	Vercel

The backend mounts routes for /api/learning, /api/ai, /api/ai/knowledge, /api/payments, /api/internal/notifications, /api/gamification and /api/admin.

Existing security foundations include authentication and role middleware, validation, rate limiting, CORS configuration, Helmet and central error handling. These are useful foundations, but they do not constitute a complete security approval.

4. Technical Audit Findings

4.1 Frontend

The application already contains routes for onboarding, diagnostic assessment, student dashboards, practice, CBT, progress, profiles, notifications, missions, subscriptions and administration. There are also multiple routes and aliases that should be reviewed for duplication.

Developer instruction: Do not delete or rename existing routes until their dependencies, navigation links and external references have been checked. A visible page or button is not proof that the underlying functionality works.

4.2 Learning engines

Existing source code indicates that Sabi already has adaptive question-selection logic, prioritisation of overdue reviews, weak-topic and recent-mistake targeting, a hybrid Blitz selection strategy, topic mastery updates, spaced-review interval updates and a basic 0–400 score-prediction function.

The architecture should be preserved and completed. The presence of these functions does not prove that all learner workflows operate correctly end to end.

4.3 AI engine

The repository contains a multi-provider AI architecture supporting Google Gemini, OpenAI, Anthropic and OpenAI-compatible providers. The AI system is disabled by default through SABI_AI_ENABLED=false. Existing documentation also indicates that some AI routes may return simulated drafts when a real provider is unavailable.

Critical distinction: Mock responses must never be presented as actual AI-generated responses.

The following capabilities require verification:

- Real provider execution.
- Task-based model routing.
- Provider fallback.
- AI usage and cost accounting.
- Budget enforcement.
- Response caching.
- Output validation.
- Consent and privacy enforcement.

- Consent revocation.
- Active CBT restrictions.
- Operator-facing model configuration.

Learning decisions such as question selection, grading, mastery calculations and CBT scoring must remain deterministic. They should not depend on an AI model making arbitrary decisions.

4.4 Academic content and question bank

A historical backend audit dated November 10 documented the following figures. These are historical figures, not a live database report, and must be refreshed before making current content-readiness decisions.

Content	Historical count
JAMB subjects	25
Topics	809
Subtopics	3,006
Total questions	3,740
Published questions	404
AI drafts	3,333
Syllabus objectives	0
Question-source records requiring provenance review	157

Critical requirements:

- Import and map official syllabus objectives.
- Validate answers and explanations.
- Verify question-to-topic mappings.
- Verify source provenance and permitted usage.
- Ensure AI-generated questions undergo human review.
- Prevent unapproved questions from entering student practice.

Content quality is a core product requirement, not an optional administrative task.

4.5 UTME score predictor

A basic prediction function exists. It uses smoothed subject accuracy and combines four subject estimates into a 0–400 score. However, this should not yet be marketed as a scientifically validated prediction.

- Test against representative examination outcomes.
- Measure prediction error.
- Establish minimum-data requirements.
- Display confidence or uncertainty.
- Avoid guaranteed-score claims.

5. Important Pull Request: PR #42

Pull request: https://github.com/sabiprep/sabi/pull/42

Title: feat(frontend): complete learner learning and AI Coach flows

This is the most important integration candidate identified during the audit. Its description reports work involving:

- A consent-aware Sabi AI Coach integrated with /api/ai/chat.
- Subject and topic context, AI credit and availability handling, consent revocation and mathematical rendering.
- Authenticated SmartPractice and Blitz continuation.
- Adaptive question selection, duplicate-question exclusion and persisted adaptive decisions.
- Atomic question appending through a database RPC.
- Published-question and provenance checks.
- CBT question flags, explicit submission and review navigation.
- Active CBT restrictions on AI explanations.
- A new learning-session migration and additional database assertions and tests.

The PR description reports passing checks for TypeScript, Vitest, production builds and disposable PostgreSQL assertions. These claims must be independently checked against the actual diff and current CI status.

Important: PR #42 remains a draft in the audit findings. It has not been approved, merged or deployed.

Before merging: review the complete code diff; compare it against the current main branch; verify the migration against hosted Supabase migration history; confirm security and permission changes; complete authenticated browser acceptance testing; and verify the frontend and API together in staging. Do not apply the migration to production without explicit approval.

6. Feature Expansion and Scope Control

Two additional pull requests require careful scope management:

- PR #34 – Parent, school and tutorial-centre portals.
- PR #35 – Expanded SABI COMMAND administration specification.

These may be useful later, but they should not take priority over completing the core learner product. The development team must not merge features simply because they are available.

Scope test: Does this materially improve learning, mastery, practice quality, examination readiness or platform reliability?

If not, it should be deferred.

7. Target System Architecture

7.1 Learning engine

Student → Onboarding → Diagnostic Assessment → Learner Profile → Learning Engine → Question Selection → Answer Submission → Mastery Update → Revision Scheduling → Recommendations → Next Learning Action.

The learning engine should continuously use learner performance to determine what the student should practise next.

7.2 AI engine

AI Task → Consent and Privacy Check → Model Router → Provider Selection → AI Execution → Output Validation → Usage and Cost Accounting → Learner.

The system must support multiple providers without tightly coupling the entire application to one AI company. The administrator should be able to manage provider availability, models, routing rules and spending controls.

8. Developer Execution Roadmap

P0 – Protect and consolidate the existing system

- Establish a verified rollback point.
- Inspect PR #42 and compare it with main.
- Identify unfinished, duplicated and disconnected implementations.

- Reconcile migration history.
- Review environment configuration.
- Identify broken routes and API integrations.
- Avoid destructive refactoring.
- Keep unfinished features out of the main learner journey.

Deliverable: A verified baseline and an approved integration plan.

P1 – Complete the core learning experience

- Onboarding and diagnostic assessment.
- SmartPractice and Blitz.
- Question feedback and mastery updates.
- Spaced repetition and recommendations.
- Progress tracking and CBT simulator.

Every learner action must persist correctly. Deliverable: A complete working learner journey from registration to personalised practice and progress tracking.

P2 – Complete the AI Coach and AI infrastructure

- Enable genuine AI provider execution.
- Implement task-based model routing and provider fallback.
- Configure consent enforcement and privacy protections.
- Verify AI credit accounting and enforce spending limits.
- Implement usage monitoring and output validation.
- Enforce active CBT restrictions.
- Provide clear error handling when AI is unavailable.

Deliverable: A reliable, measurable and cost-controlled AI tutoring system.

P3 – Improve academic content readiness

- Refresh question-bank statistics.
- Complete syllabus-objective ingestion.
- Verify topic and subtopic mapping.
- Review question provenance.
- Validate answers and explanations.
- Approve questions before publication.
- Test question selection across subjects and topics.

Deliverable: A reliable, traceable and academically useful question bank.

P4 – Security and launch readiness

- Verify user-data isolation.
- Test authentication and authorisation.
- Review API rate limits.
- Verify database permissions.
- Reconcile and test migrations.
- Test backup and restoration procedures.
- Complete staging acceptance tests.

- Verify production frontend/API compatibility.
- Configure monitoring and error reporting.

Deliverable: A stable and secure release candidate.

9. Mandatory End-to-End Tests

Test	Expected outcome
Student registration	Account is created and authenticated
Onboarding	Preferences and goals persist
Diagnostic	Answers and results persist
SmartPractice	Questions reflect learner needs
Blitz	Practice continues adaptively
Answer submission	Mastery and review data update
Spaced repetition	Revision intervals are respected
Recommendations	Relevant next actions appear
CBT	Timer, navigation and submission work
CBT review	Answered and flagged states are correct
AI Coach	Real provider returns validated output
AI consent	Revocation prevents further processing
CBT AI lock	AI tutoring is blocked during an active CBT
Cost accounting	AI usage is recorded correctly
Access control	Users cannot access another user's private data
Question moderation	Unapproved questions remain unavailable

Tests must cover normal use, failure cases, expired sessions, retries and concurrent requests where applicable.

10. Non-Negotiable Development Rules

13. Do not rebuild the platform from scratch.
14. Preserve existing working code.
15. Do not delete functionality without dependency analysis.
16. Do not introduce unnecessary features.
17. Do not merge unfinished work simply to increase feature count.
18. Do not apply production database migrations without approval.
19. Never expose secrets in frontend code or source control.
20. Never count mock AI output as genuine AI execution.
21. Never count a page or button as a completed feature without testing its underlying workflow.
22. Persist important learner actions and learning progress.
23. Maintain appropriate user-data isolation.
24. Keep deterministic learning and examination logic outside the AI model.
25. Test migrations before production application.

26. Document material architectural and database changes.
27. Obtain approval before destructive operations, production changes or major scope expansion.

11. Final Developer Directive

The objective is not to make Sabi bigger. The objective is to make Sabi work exceptionally well.

The platform already has a meaningful foundation. The development team must focus on completing the original product vision rather than continuously introducing new features.

PRESERVE → CONSOLIDATE → COMPLETE → VERIFY → LAUNCH

Every development task should move Sabi closer to becoming a dependable, intelligent and genuinely adaptive JAMB preparation platform.

12. Audit Scope and Limitations

This audit involved reviewing repository source files, project documentation, migrations, branch information and pull-request information.

The following activities were not performed during the audit:

- Local execution of the application.
- Independent execution of the full test suite.
- Live Supabase database inspection.
- Authenticated browser acceptance testing.
- Production deployment verification.
- Production database modification.

No repository files were changed, no pull request was merged, no production data was modified and no deployment was performed.

The findings should therefore be treated as a source-code and documentation audit, with the verification activities above forming part of the next engineering phase.